Digital Signal Processing in VLSI Design

Homework 3
Interpolator

Name : 陳映宬
Department : Electrical Engineering
Student ID : B11107027
Instructor : 蔡佩芸

April 13 , 2026

Contents

1 Interpolator Function Verification in Over-Sampled Cases (Step 1) 2
1.1 Linear & Second-Order Polynomial Interpolator Results 2
1.2 Error Comparison 3

2 Interpolator Function Verification in Typical Cases (Step 2) 4
2.1 Linear & Piecewise Parabolic Interpolator Results 4
2.2 Error Comparison 5

3 Bit-True Model Construction (Step 3) 6
3.1 BF16 Format Overview 6
3.2 BF16 Addition and Multiplication Implementation 6
3.3 Verification Test Cases 7

4 Performance Degradation Analysis with BF16 Arithmetic (Step 4) 8
4.1 Error Analysis: Second-Order Farrow Interpolator with BF16 Operations . 8

5 Hardware Reduction (Step 5) 10
5.1 Original Piecewise Parabolic Block Diagram 10
5.2 Hardware Sharing Approaches 10
5.3 Revised Block Diagram 10

6 Hardware Implementation (Step 6) 12
6.1 Behavioral Simulation and Latency 12
6.2 Circuit Synthesis and Critical Path Analysis 13
6.3 Post-Synthesis Simulation 13
6.4 Error Plot: Hardware vs. MATLAB Floating-Point 14

1 Interpolator Function Verification in Over-Sampled Cases (Step 1)

The input signal is defined as

$$x_1[m] = \cos\left(2\pi \frac{mT_s}{10T_s} + \frac{\phi}{2}\right) + j\sin\left(2\pi \frac{mT_s}{10T_s} + \frac{\phi}{2}\right),$$

where $\phi = \mathrm{mod}(d, 2)$ and d is the last digit of the student ID ($d = 7$, $\phi = 1$). The sampling frequency is 10 times the signal frequency. Interpolation is performed in the region $10 \le m \le 20$ with $\mu = 0, \frac{1}{8}, \frac{2}{8}, \ldots, \frac{7}{8}$.

1.1 Linear & Second-Order Polynomial Interpolator Results

Linear interpolation formula:

$$x(m+\mu) = \mu\, x(m+1) + (1-\mu)\, x(m). \tag{1}$$

Second-order polynomial interpolation formula:

$$x(m+\mu) = C_0\, x(m) + C_{-1}\, x(m+1) + C_{-2}\, x(m+2), \tag{2}$$

where

$$\begin{cases} C_0 = (1-\mu)(2-\mu)/2 \\ C_{-1} = \mu(2-\mu) \\ C_{-2} = -\mu(1-\mu)/2 \end{cases}$$

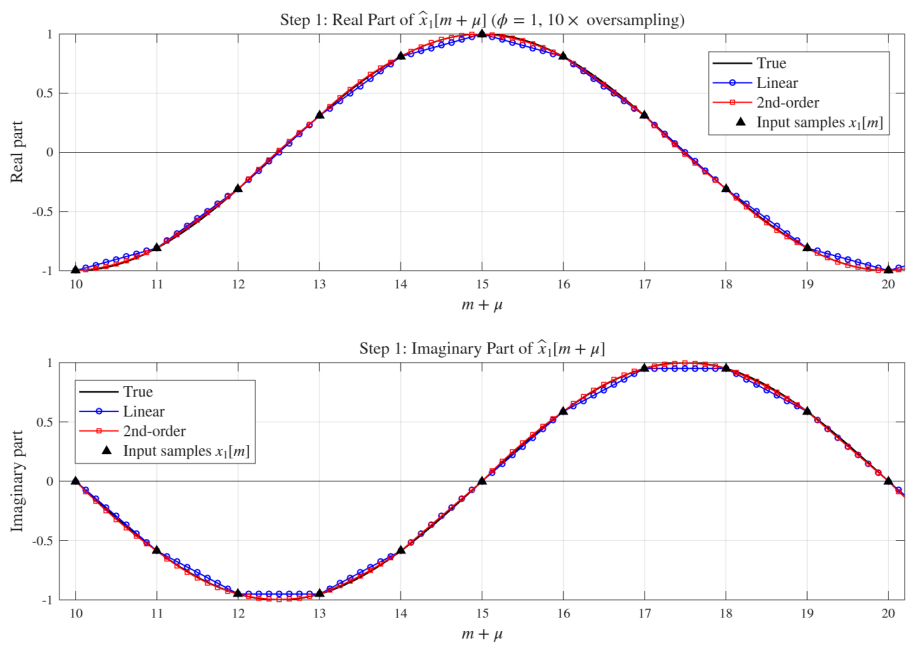

Figure 1: Real and imaginary parts: linear vs. second-order polynomial vs. true values (over-sampled case).

Analysis: Both interpolators track the true signal well at 10× oversampling. The second-order polynomial shows visibly smaller deviation at signal peaks and troughs.

1.2 Error Comparison

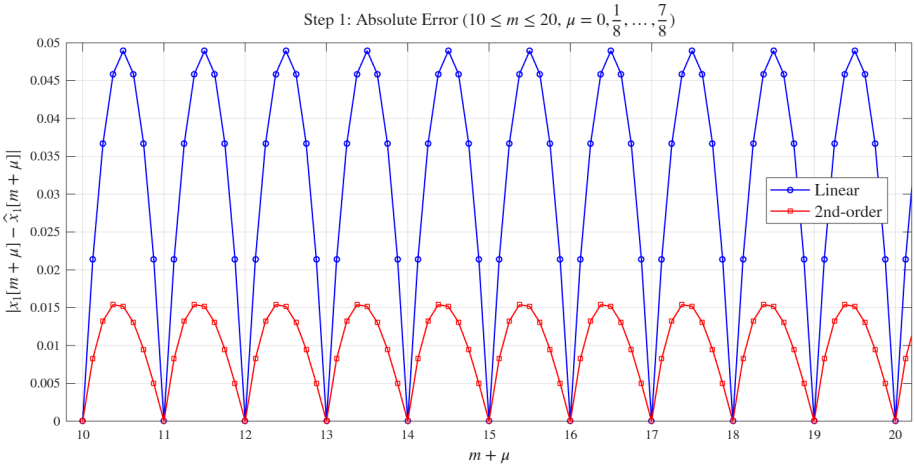

Figure 2: Absolute error comparison between linear and second-order polynomial interpolators (over-sampled case).

Table 1: Error summary for over-sampled case.

Method	**Mean |error|**	**Max |error|**
Linear interpolator	3.2103e-02	4.8943e-02
Second-order polynomial	9.9507e-03	1.5393e-02

The second-order polynomial achieves $\sim 3.2\times$ lower max error, as a quadratic fit better captures local curvature than a straight-line approximation.

2 Interpolator Function Verification in Typical Cases (Step 2)

The input signal is defined as

$$x_2[m] = 2^{-20}\left[\cos\left(2\pi\frac{mT_s}{4T_s} + \frac{\phi}{3}\right) + j\sin\left(2\pi\frac{mT_s}{4T_s} + \frac{\phi}{3}\right)\right],$$

where $\phi = 1$ (same as Step 1). The sampling frequency is 4 times the signal frequency. Interpolation is performed in the region $4 \le m \le 8$ with $\mu = 0, \frac{1}{8}, \frac{2}{8}, \ldots, \frac{7}{8}$.

2.1 Linear & Piecewise Parabolic Interpolator Results

Linear interpolation formula: Same as Eq. (1).

Piecewise parabolic interpolation formula ($\alpha = 0.5$):

$$x(m+\mu) = C_1\, x(m-1) + C_0\, x(m) + C_{-1}\, x(m+1) + C_{-2}\, x(m+2), \tag{3}$$

where

$$\begin{cases} C_1 = -\alpha\mu + \alpha\mu^2 \\ C_0 = 1 + (\alpha - 1)\mu - \alpha\mu^2 \\ C_{-1} = (\alpha + 1)\mu - \alpha\mu^2 \\ C_{-2} = -\alpha\mu + \alpha\mu^2 \end{cases}$$

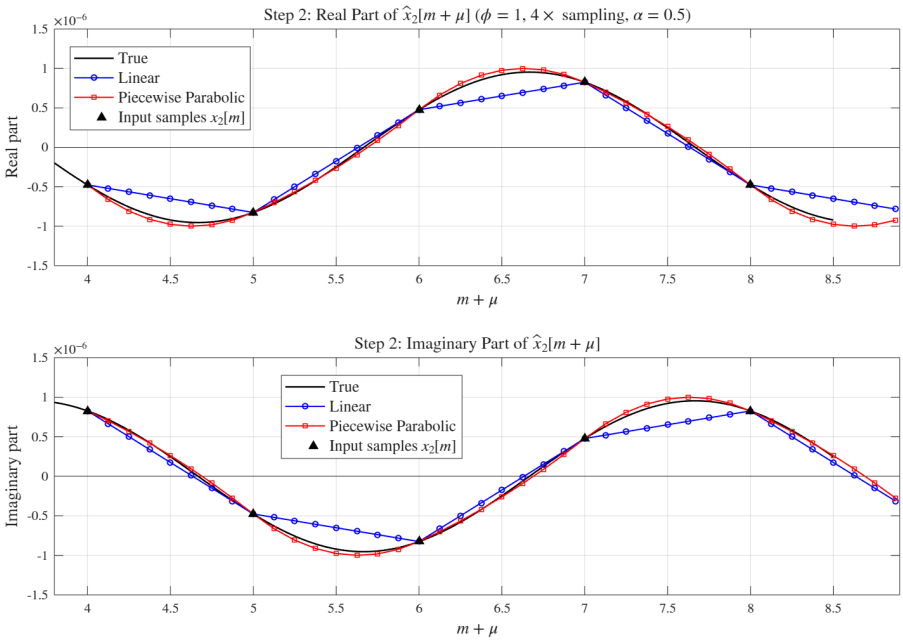

Figure 3: Real and imaginary parts: linear vs. piecewise parabolic vs. true values (typical case).

Analysis: Both interpolators reproduce the signal shape at 4× oversampling. The piecewise parabolic interpolator fits local curvature using four sample points, yielding visibly closer agreement with the true value.

2.2 Error Comparison

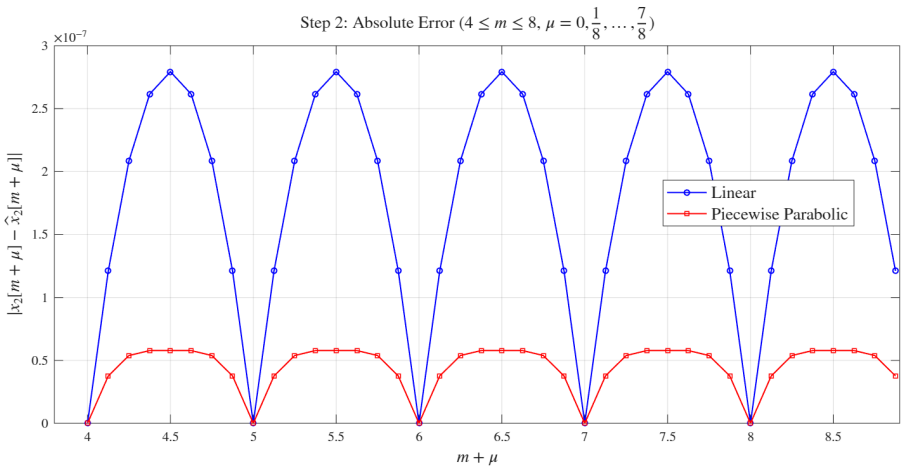

Figure 4: Absolute error comparison between linear and piecewise parabolic interpolators (typical case).

Table 2: Error summary for typical case.

Method	**Mean |error|**	**Max |error|**
Linear interpolator	1.8271e-07	2.7932e-07
Piecewise parabolic	4.4554e-08	5.7850e-08

The piecewise parabolic interpolator achieves approximately 4.1× lower error than the linear interpolator in the typical case. Although the 4× oversampling ratio is lower than Step 1, both methods achieve very small absolute errors due to the small signal amplitude (2^{-20}). The piecewise parabolic interpolator uses four sample points and a second-order polynomial, giving it better local curvature approximation and thus smaller error.

3 Bit-True Model Construction (Step 3)

3.1 BF16 Format Overview

The BF16 (Brain Float 16) format is a 16-bit floating-point representation:

- **Sign**: 1 bit
- **Exponent**: 8 bits (same as IEEE 754 single-precision, bias = 127)
- **Mantissa**: 7 bits (truncated from the 23-bit single-precision mantissa)

The value of a normalized BF16 number is:

$$v = (-1)^s \times 2^{e-127} \times \left(1 + \text{mantissa} \times 2^{-7}\right).$$

Simplifications per lab specification:

- Subnormal numbers ($e = 0$, mantissa $\neq 0$): treated as **zero** (flush-to-zero).
- NaN and infinity: not handled.
- All intermediate results use **truncation** (not rounding) to avoid renormalization overhead.

3.2 BF16 Addition and Multiplication Implementation

BF16 multiplication computes the result sign as $S_r = S_a \oplus S_b$ and the result exponent as $E_r = E_a + E_b - 127$. The 8-bit mantissas (with implicit leading 1) are multiplied to a 14–15 bit product, then right-shifted by 7 or 8 bits with **truncation** to recover the 7-bit fraction. Subnormal inputs and underflow results are flushed to zero.

BF16 addition first aligns the smaller operand by right-shifting its mantissa by the exponent difference, applying **truncation** to dropped bits. A signed integer addition is then performed, followed by re-normalization (shift right on overflow, shift left on leading zeros). Subnormal inputs and underflow results are also flushed to zero.

Listing 1: BF16 multiplication and addition — key logic (MATLAB).

```
% --- Multiplication key steps ---
E_r = E_a + E_b - 127;            % exponent sum (remove one bias)
P   = (128+F_a) * (128+F_b);      % 8x8 mantissa product
if P >= 32768
    M_r = floor(P/256); E_r = E_r+1;  % normalise: implicit 1 at bit 15
else
    M_r = floor(P/128);                % normalise: implicit 1 at bit 14
end                                    % truncation applied by floor()

% --- Addition key steps ---
shift = E_a - E_b;
M_b   = floor(M_b / 2^shift);     % align smaller operand (truncation)
val   = (1-2*S_a)*M_a + (1-2*S_b)*M_b;  % signed addition
if M_r >= 256
    M_r = floor(M_r/2); E_r = E_r+1;    % overflow: shift right
end
% underflow: if E_r <= 0 -> result = 0
```

3.3 Verification Test Cases

Four test cases are used to verify the correctness of the BF16 bit-true model. Each operand is specified in the format `S/EEEE_EEEE/MMMM_MMM` (sign / exponent / mantissa).

Listing 2: Step 3 BF16 bit-true test results.

```
Case (a) : Addition
  Op1 : 1/1000_0011/0000_000  =  -16
  Op2 : 0/1000_1011/0000_011  =  +4192
  Res : 0/1000_1011/0000_011  =  +4192

Case (b) : Addition
  Op1 : 1/0000_0001/0000_011  =  -1.20304e-38
  Op2 : 0/0000_0001/1111_010  =  +2.29589e-38
  Res : 0/0000_0000/0000_000  =  +0

Case (c) : Multiplication
  Op1 : 1/0000_0010/1100_000  =  -4.11423e-38
  Op2 : 0/0111_1100/0000_110  =  +0.130859
  Res : 0/0000_0000/0000_000  =  +0

Case (d) : Multiplication
  Op1 : 0/0110_0011/1011_110  =  +6.46105e-09
  Op2 : 0/1001_0011/0101_000  =  +1.37626e+06
  Res : 0/0111_1000/0010_001  =  +0.0088501
```

Analysis:

- **Case (a) — Addition, large magnitude difference:** Op2 (+4192) is $4192/16 = 262 \approx 2^{8.03}$ times larger than |Op1| (16). During exponent alignment, Op1 must be right-shifted by 8 bits, which exceeds the 7-bit mantissa width, so Op1 is completely lost after truncation. Result = +4192 (correct BF16 behavior).
- **Case (b) — Addition, result underflows to zero:** The true sum $\approx +1.09 \times 10^{-38}$ is below the smallest normal BF16 value $2^{-126} \approx 1.175 \times 10^{-38}$. Per the flush-to-zero rule, the result is set to **0**.
- **Case (c) — Multiplication, result underflows to zero:** The true product $\approx -5.38 \times 10^{-39}$ is also below 2^{-126}. Flush-to-zero gives result = **0**.
- **Case (d) — Multiplication, normal result:** $6.461 \times 10^{-9} \times 1.376 \times 10^{6} \approx 8.89 \times 10^{-3}$. After BF16 truncation the result is +0.0088501, within the normal range.

4 Performance Degradation Analysis with BF16 Arithmetic (Step 4)

4.1 Error Analysis: Second-Order Farrow Interpolator with BF16 Operations

The second-order Farrow structure is used with the over-sampled signal $x_1[m]$. All arithmetic operations (multiply and add) are replaced by BF16 equivalents.

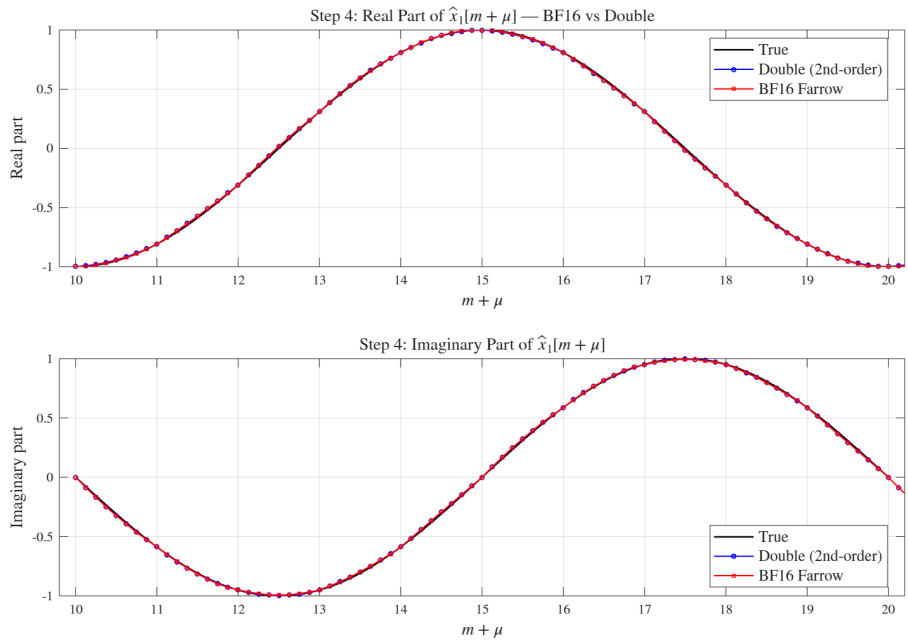

Figure 5: Real and imaginary parts: double-precision vs. BF16 Farrow interpolator vs. true values.

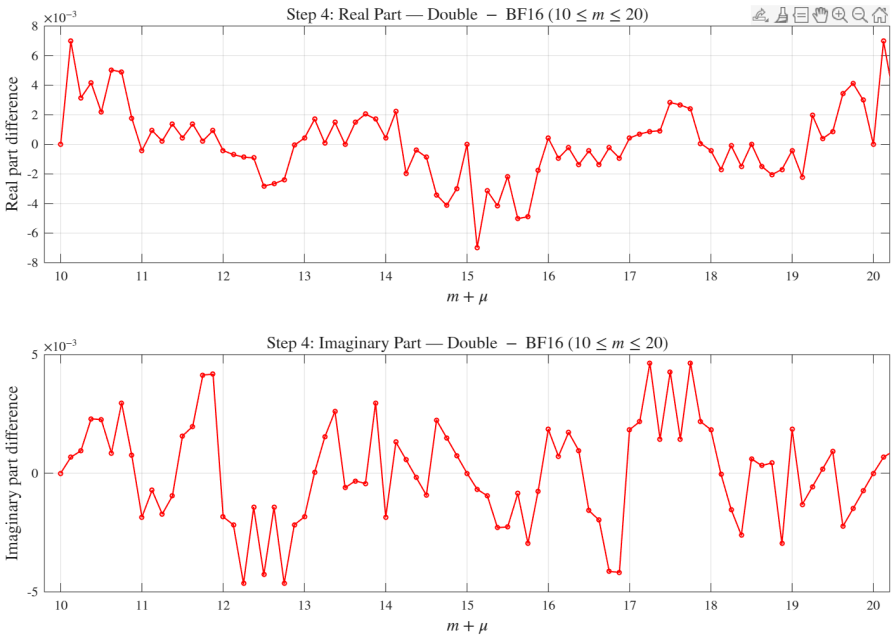

Figure 6: Difference between double-precision and BF16 Farrow outputs (real and imaginary parts).

Table 3: Difference between double-precision and BF16 Farrow outputs.

	Max |diff|	**Mean |diff|**
Real part	6.9735e-03	1.9166e-03
Imaginary part	4.6444e-03	1.7162e-03

Analysis: The maximum difference between double-precision and BF16 Farrow outputs is approximately 6.97×10^{-3}. BF16 has only 7 mantissa bits (compared to 52 bits for double-precision), giving a precision of $2^{-7} \approx 7.8 \times 10^{-3}$ per unit. The observed error is consistent with this quantization granularity. For a communication system, this level of degradation may be acceptable depending on the SNR requirements; however, it is significantly coarser than the double-precision baseline.

5 Hardware Reduction (Step 5)

5.1 Original Piecewise Parabolic Block Diagram

[Figure placeholder: step5$_{o}riginal_{b}lock.png$]

Figure 7: Original block diagram of the piecewise parabolic interpolator (Farrow structure, $\alpha = 0.5$).

[TODO: Describe the resource count of the original design: number of D flip-flops, multipliers, and adders.]

5.2 Hardware Sharing Approaches

Two hardware reduction approaches are considered:

Approach 1 — Original Design: [TODO: Describe the original design and its resource usage.]

Approach 2 — [TODO: Name of simplified design]: [TODO: Describe the hardware sharing technique applied (e.g., shared $0.5\times$ scaling via exponent shift, reuse of $\pm 0.5\,x$ terms across v_0, v_1, v_2) and quantify the savings in multipliers, adders, and flip-flops.]

Table 4: Hardware cost comparison between original and simplified designs.

Resource	Original	Simplified
D Flip-Flops	TODO	TODO
Multipliers	TODO	TODO
Adders	TODO	TODO

5.3 Revised Block Diagram

[Figure placeholder: step5$_{r}evised_{b}lock.png$]

Figure 8: Revised block diagram after hardware reduction.

[TODO: Explain the revised design and confirm it produces identical outputs to the original.]

6 Hardware Implementation (Step 6)

6.1 Behavioral Simulation and Latency

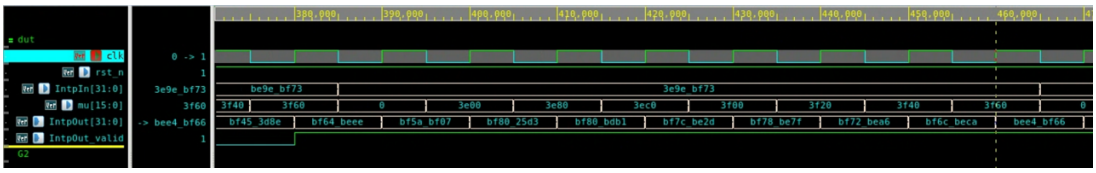

Figure 9: Behavioral simulation timing diagram — first 8 outputs.

Figure 10: Behavioral simulation timing diagram — last 8 outputs.

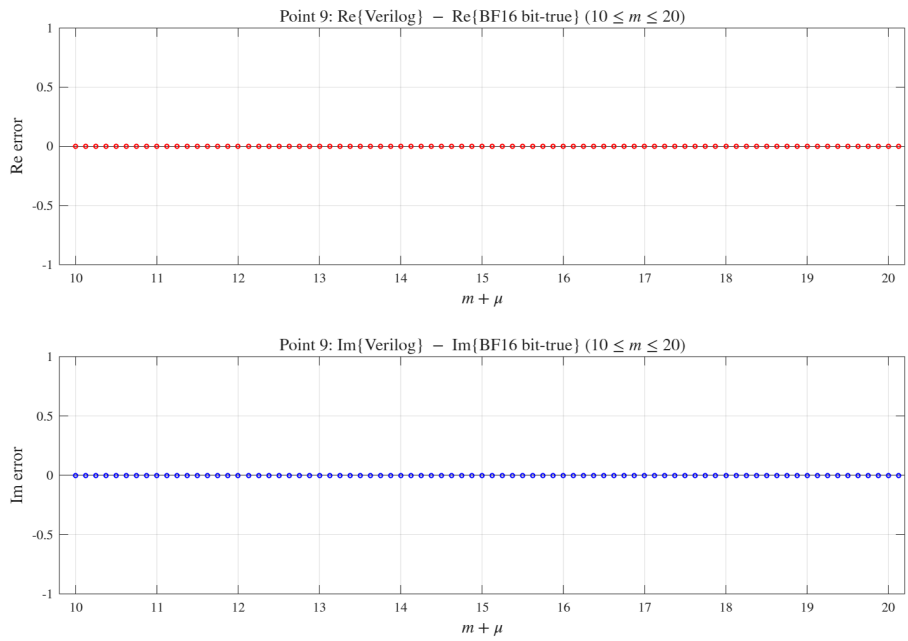

Figure 11: Point 9: error between Verilog behavioral simulation and BF16 bit-true model (all zeros confirms bit-exact match).

Latency Explanation: [TODO: State the overall pipeline latency (in clock cycles) and explain which pipeline registers contribute to it.]

6.2 Circuit Synthesis and Critical Path Analysis

A) Theoretical Critical Path: [TODO: Describe the theoretical critical path of the Farrow-structure interpolator (number and type of operations on the longest combinational path).]

B) Post-Synthesis Critical Path: [TODO: Report the actual critical path after synthesis: resource types (DSP48E1, CARRY4, etc.), number of stages, and total delay (ns). State the timing constraint T_{clk}.]

[Figure placeholder: `step6`$_{t}iming_{r}eport.png$]

Figure 12: Post-synthesis timing report of the critical path.

Timing Constraint: The clock period is set to T_{clk} = [TODO: XX ns (XX MHz)], the minimum value before the synthesis tool reports a timing violation.

6.3 Post-Synthesis Simulation

Figure 13: Post-synthesis simulation timing diagram — first 8 outputs.

Figure 14: Post-synthesis simulation timing diagram — last 8 outputs.

[Figure placeholder: `step6`$_{p}oint8_{p}ostsyn.png$]

Figure 15: Point 8 (post-synthesis): BF16 hardware output vs. double-precision true value.

6.4 Error Plot: Hardware vs. MATLAB Floating-Point

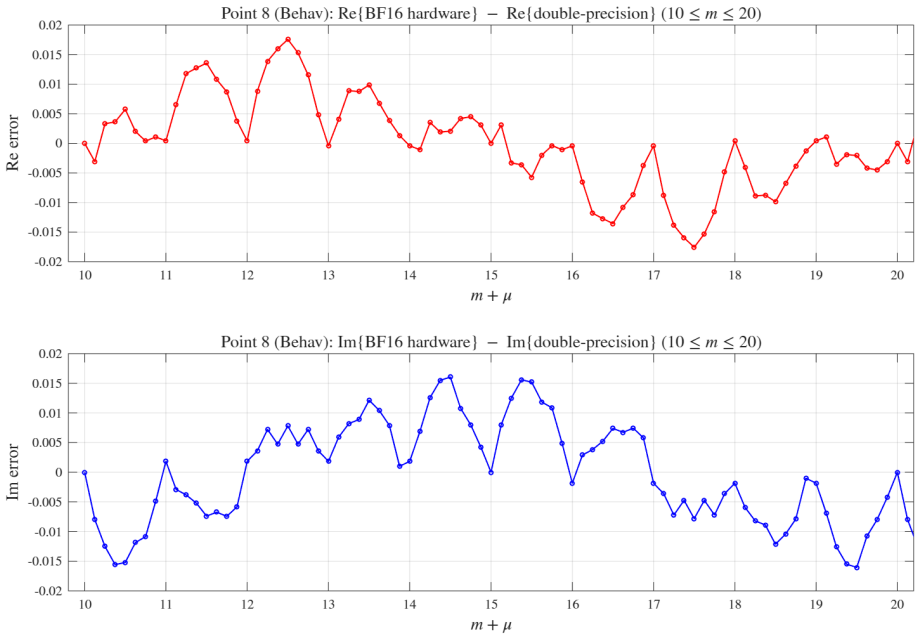

Figure 16: Point 8 (behavioral): BF16 hardware output vs. double-precision true value.

The Verilog behavioral simulation produces outputs that are **bit-exact** with the BF16 bit-true MATLAB model: all 176 outputs (88 real + 88 imaginary) have zero error (Point 9 verification). Compared to the double-precision true value, the BF16 hardware output has a maximum error of 1.7578×10^{-2} (real part) and 1.6048×10^{-2} (imaginary part), which is consistent with the BF16 precision limit ($2^{-7} \approx 7.8 \times 10^{-3}$) observed in Step 4.